



الهيئة الاتحادية للهوية
والجنسية والجمارك وأمن المنافذ
FEDERAL AUTHORITY FOR IDENTITY,
CITIZENSHIP, CUSTOMS & PORT SECURITY

ID Card Toolkit Developer Guide

Objective-C & Swift

Version 1.25

March 2026

CONFIDENTIAL

Table of Contents

Revision History	4
Abbreviations	5
1 Introduction	6
1.1 Purpose	6
2 Development Environment Setup	7
2.1 Prerequisites	7
2.2 Environment Setup for iOS	7
2.3 Environment Setup for macOS	7
3 Developing Your First Program	8
3.1 Step 1: Initialize the Toolkit	8
3.2 Step 2: List Connected Readers	8
3.3 Step 3: Connect to a Card Reader	8
3.4 Step 4: Read Public Data	8
3.5 Step 5: Disconnect	9
4 Toolkit API Reference	10
4.1 Class: Toolkit	10
4.1.1 Constructor: Toolkit	10
4.1.2 Method: GetToolkitVersion	10
4.1.3 Method: ListReaders	10
4.1.4 Method: GetReaderWithEmiratesId	11
4.1.5 Method: PrepareRequest	11
4.1.6 Method: RegisterDevice	11
4.1.7 Method: GetDeviceId	12
4.1.8 Method: GetDataProtectionKey	12
4.1.9 Method: ParseMRZData	12
4.1.10 Method: GetConfigCertificateExpiryDate	13
4.1.11 Method: GetLicenseExpiryDate	13
4.1.12 Method: setNfcTag	13
4.2 Class: CardReader	13
4.2.1 Method: Connect	14
4.2.2 Method: Disconnect	14
4.2.3 Method: GetInterfaceType	14
4.2.4 Method: SetNfcAuthenticationParameters (Card Attributes)	14
4.2.5 Method: SetNfcAuthenticationParameters (MRZ)	14
4.2.6 Method: CheckCardStatus	15
4.2.7 Method: GetCSN	15
4.2.8 Method: GetCardVersion	15
4.2.9 Method: ReadPublicData	15
4.2.10 Method: ReadPublicDataEF	16
4.2.11 Method: ParseEFData	16
4.2.12 Method: PrepareRequest (CardReader)	17
4.2.13 Method: GetPkiCertificates	17
4.2.14 Method: AuthenticatePki	17
4.2.15 Method: SignData	17
4.2.16 Method: SignChallenge	18

4.2.17 Method: GetFingerData	18
4.2.18 Method: AuthenticateBiometricOnServer	18
4.2.19 Method: AuthenticateCardAndBiometric	18
4.2.20 Method: ResetPin	19
4.2.21 Method: ResetPinWithoutAuthenticateBiometric	19
4.2.22 Method: UnblockPin	19
4.2.23 Method: ReadFamilyBookData	19
4.2.24 Method: UpdateData	19
4.2.25 Method: ReadData	20
4.2.26 Method: VerifySignature	20
4.3 Class: SignatureValidator	20
4.3.1 Constructor: SignatureValidator	20
4.3.2 Method: ValidateToolkitResponse	21
5 Advanced Digital Signature API Reference	22
5.1 Class: CardReader (Digital Signature Methods)	22
5.1.1 Method: PadesSign	22
5.1.2 Method: PadesVerify	22
5.1.3 Method: XadesSign	22
5.1.4 Method: XadesVerify	22
5.1.5 Method: CadesSign	23
5.1.6 Method: CadesVerify	23

Revision History

Version	Date	Description of Changes
1.0	26th Feb, 2018	Release Version 1.0.0
1.1	30th Mar, 2018	Release Version 1.0.3
1.2	28th May, 2018	Release Version 1.0.5
1.3	12th Jun, 2018	Release Version 1.0.6
1.4	28th Sep, 2018	Release Version 1.0.8
1.5	13th Sep, 2018	Release Version 1.0.9
1.6	16th Feb, 2019	Release Version 1.0.13
1.7	20th Mar, 2019	Release Version 1.1.0
1.8	03rd Apr, 2019	Release Version 1.2.0
1.9	13th May, 2019	Release Version 1.3.0
1.10	07th Jun, 2019	Release Version 1.4.0
1.11	17th Jun, 2019	Release Version 1.4.1
1.12	31st Jul, 2019	Release Version 1.4.3; Included APIs for Card Update functionality.
1.13	18th Dec, 2019	Release Version 1.4.4
1.14	04th Mar, 2020	Release Version 1.4.5
1.15	03rd Jan, 2021	Release Version 1.4.8
1.16	13th May, 2021	Release Version 1.4.9
1.17	02nd Nov, 2021	Release Version 2.0.3
1.18	23rd Jun, 2022	Release Version 2.0.5
1.19	01st Nov, 2022	Release Version 2.0.6
1.20	09th Mar, 2023	Release Version 2.0.7
1.21	02nd Jan, 2024	Release Version 3.0.0
1.22	14th Mar, 2024	Release Version 3.0.1-r1
1.23	07th May, 2024	Release Version 3.0.1-r5
1.24	22nd May, 2025	Release Version 3.0.3-r1
1.25	Mar 2026	Toolkit v3.0.5. Improved code samples.

Abbreviations

Abbreviation	Description
CA	Certificate Authority
CAeS	CMS Advanced Electronic Signatures
CMS	Cryptographic Message Syntax
EF	Elementary File
ICP	Federal Authority for Identity, Citizenship, Customs & Port Security
IDE	Integrated Development Environment
iOS	iPhone Operating System
MRZ	Machine Readable Zone
NFC	Near Field Communication
PAeS	PDF Advanced Electronic Signatures
PDF	Portable Document Format
SP	Service Provider
VG	Validation Gateway
XAdES	XML Advanced Electronic Signatures
XML	Extensible Markup Language

1. Introduction

The Federal Authority for Identity, Citizenship, Customs & Port Security (ICP) developed the ID Card Toolkit to address the requirements of service providers (SP) to integrate, into their business applications: Identification, Authentication, Digital Signature and Non-repudiation services, around the capabilities of the Emirates ID Card. The Toolkit is comprised of a number of Software Development Kits (SDK) supporting different programming languages and platforms.

1.1. Purpose

This document is a developer's guide to support the development of Objective-C and Swift applications (iOS and macOS) that integrate the ICP ID Card Toolkit SDK.

2. Development Environment Setup

2.1. Prerequisites

- Xcode 15 or later (full install required)
- iOS 14.0 or later
- macOS 10.11.5 or above
- ICP ID Card Toolkit SDK for iOS and Mac

2.2. Environment Setup for iOS

1. Create an iOS project in Xcode and set the minimum deployment target.
2. Add the Toolkit SDK framework (`Toolkit.framework`) and plugin frameworks in **Link Binary with Libraries** and **Embedded Binaries**.
3. Configure the SDK to be copied to the application bundle on build.
4. Add system frameworks for external and Bluetooth device communication:
 - `ExternalAccessory.framework`
 - `CoreBluetooth.framework`
5. Add the required background mode permissions:
 - External accessory communication
 - Uses Bluetooth LE accessories
 - Acts as a Bluetooth LE accessory
6. Add supported accessory protocols in `Info.plist`:
 - Feitian Smartcard plugin: `com.ftsafe.iR301`, `com.ftsafe.bR301`
7. Add **Application Transport Security Settings > Allow Arbitrary Loads = YES**.
8. Set **ENABLE_BITCODE = NO** in project **Build Settings**.

2.3. Environment Setup for macOS

1. Create a macOS project in Xcode with minimum deployment target 10.10.
2. Add the Toolkit SDK framework and plugin frameworks.
3. Install CCID drivers for smartcard reader access.
4. Add **Application Transport Security Settings > Allow Arbitrary Loads = YES**.
5. Disable **App Sandbox** in project **Capabilities** for file operations.
6. Set **ENABLE_BITCODE = NO** in project **Build Settings**.

3. Developing Your First Program

The following example shows the sequence of API calls for reading public data. Sample Objective-C and Swift programs are provided with the SDK.

3.1. Step 1: Initialize the Toolkit

Objective-C:

```
Toolkit *toolkit = [[Toolkit alloc] initWithToolkit:YES  
                  configParams:configParams];
```

Swift:

```
var toolkit: Toolkit?  
toolkit = Toolkit(true, configParams: configParams!)
```

3.2. Step 2: List Connected Readers

Objective-C:

```
NSArray *readers = [toolkit listReaders];
```

Swift:

```
var readers = NSArray()  
readers = toolkit!.listReaders()
```

3.3. Step 3: Connect to a Card Reader

Objective-C:

```
CardReader *cardreader = (CardReader *)[readers objectAtIndex:0];  
[cardreader connect];
```

Swift:

```
var cardreader: CardReader? = readers[0] as? CardReader  
cardreader?.connect()
```

Note: Use `getReaderWithEmiratesId` to directly access a reader with a card inserted:

Objective-C: `cardreader = [toolkit getReaderWithEmiratesId];`

Swift: `cardreader = toolkit?.getReaderWithEmiratesId()`

3.4. Step 4: Read Public Data

Objective-C:


```
CardPublicData *cardPublicData =  
    [cardreader readPublicData:requestId  
        readnonModifiableData:TRUE  
        readModifiableData:TRUE  
        readPhotography:TRUE  
        readSignatueImage:TRUE  
        readAddress:TRUE];
```

Swift:

```
let cardPublicData = cardreader?.readPublicData(  
    requestId!,  
    readNonModifiableData: true,  
    readModifiableData: true,  
    readPhotography: true,  
    readSignatueImage: true,  
    readAddress: true)
```

3.5. Step 5: Disconnect

Objective-C: [cardreader disconnect];

Swift: cardreader?.disconnect()

4. Toolkit API Reference

4.1. Class: Toolkit

This class provides methods required to initialize the Toolkit and manage connected smartcard readers.

4.1.1. Constructor: Toolkit

Initializes the Toolkit context and prepares the SDK for use.

Objective-C:

```
-(instancetype)initWithToolkit:(BOOL)inProcessMode  
                      configParams:(NSString *)configParams;
```

Swift:

```
init(_ inProcessMode: Bool, configParams: String?)
```

Parameter	Type	Description
inProcessMode	BOOL	When YES/true, the Toolkit runs in-process. When NO/false, requests are forwarded to the Agent.
configParams	NSString *	Configuration parameters as a string. Pass nil to use the default config_ap file. Refer to the Programmer's Reference for details.

Throws: ToolkitException on error.

4.1.2. Method: GetToolkitVersion

Returns the Toolkit version string.

Objective-C:

```
-(NSString *)getToolkitVersion;
```

Swift:

```
func getToolkitVersion() -> String?
```

Return: NSString – version in major.minor.patch format. **Throws:** ToolkitException on error.

4.1.3. Method: ListReaders

Lists all compatible smartcard readers connected to the system.

Objective-C:

```
-(NSArray *)listReaders;
```

Swift:

```
func listReaders() -> [Any]?
```

Return: NSArray of CardReader objects. Empty array if none found. **Throws:** ToolkitException on error.

4.1.4. Method: GetReaderWithEmiratesId

Returns the first CardReader with an Emirates ID Card inserted.

Objective-C:

```
-(CardReader *)getReaderWithEmiratesId;
```

Swift:

```
func getReaderWithEmiratesId() -> CardReader?
```

Return: CardReader object. **Throws:** ToolkitException on error.

4.1.5. Method: PrepareRequest

Establishes a secure context for Toolkit service execution. The returned Request Handle is used to encode sensitive information (User ID, Password, PIN).

Objective-C:

```
-(NSString *)prepareRequest:(NSString *)requestId;
```

Swift:

```
func prepareRequest(_ requestId: String) -> String?
```

Parameter	Type	Description
requestId	NSString *	Application-generated request identifier. Refer to the Programmer's Reference for guidelines.

Return: NSString – 8-byte request handle in Base64 format. **Throws:** ToolkitException on error.

4.1.6. Method: RegisterDevice

Registers a device with the VG. PrepareRequest must be called first.

Objective-C:

```
-(RegisterDeviceResponse *)registerDevice:(NSString *)encodedUserId
                                encodedPassword:(NSString *)encodedPassword
                                deviceReferenceId:(NSString *)deviceReferenceId;
```

Parameter	Type	Description
encodedUserId	NSString *	SP User ID, encrypted and Base64-encoded per the Programmer's Reference.
encodedPassword	NSString *	Password, encrypted and Base64-encoded per the Programmer's Reference.
deviceReferenceId	NSString *	Unique device ID generated by the SP.

Return: RegisterDeviceResponse – contains device registration ID. The getXmlString method returns the XML response. **Throws:** ToolkitException on error.

4.1.7. Method: GetDeviceId

Returns the unique device identifier for deregistration via ICP REST API.

Objective-C:

```
-(NSString *)getDeviceId;
```

Return: NSString – the device identifier. **Throws:** ToolkitException on error.

4.1.8. Method: GetDataProtectionKey

Returns the VG public key for encoding sensitive parameters.

Objective-C:

```
-(DataProtectionKey *)getDataProtectionKey;
```

Return: DataProtectionKey object with methods:

- getPublicKey / getKeyLength – public key bytes and length
- getModulus / getModulusLength – RSA modulus and length
- getExponent / getExponentLength – RSA exponent and length

Throws: ToolkitException on error.

4.1.9. Method: ParseMRZData

Parses an MRZ string and retrieves the corresponding attributes.

Objective-C:

```
-(void)parseMRZData:(NSString *)mrz;
```

Parameter	Type	Description
mrz	NSString *	The MRZ string to parse.

After calling, access parsed attributes via: `getMrzCardnumber`, `getMrzIdnumber`, `getMrzDocument_type`, `getMrzIssued_country`, `getMrzDate_of_birth`, `getMrzGender`, `getMrzNationality`, `getMrzFullname`, `getMrzCard_expiry_date`. **Throws:** `ToolkitException` on error.

4.1.10. Method: `getConfigCertificateExpiryDate`

Returns certificate expiry dates for configuration files.

Objective-C:

```
-(void)getConfigCertificateExpiryDate;
```

After calling, access dates via: `getConfig_vg_cert_expiry`, `getConfig_lv_cert_expiry`, `getServer_tls_cert_expiry`, `getConfig_ag_cert_expiry`, `getLicense_expiry`. **Throws:** `ToolkitException` on error.

4.1.11. Method: `getLicenseExpiryDate`

Returns the SDK license expiry date.

Objective-C:

```
-(NSString *)getLicenseExpiryDate;
```

Return: `NSString` – the license expiry date. **Throws:** `ToolkitException` on error.

4.1.12. Method: `setNfcTag`

Sets the NFC communication session and tag identifier for the Toolkit. Used on iOS for NFC card access.

Objective-C:

```
-(void)setNfcTag:(id)session tag:(id>tag;
```

Parameter	Type	Description
session	id	NFC tag reader session from <code>tagReaderSession</code> callback (iOS NFC framework).
tag	id	NFC tag from <code>tagReaderSession</code> callback.

Throws: `ToolkitException` on error.

4.2. Class: `CardReader`

This class provides all methods required to perform operations with the Emirates ID Card after a successful connect operation.

4.2.1. Method: Connect

Establishes a connection to the ID card in the reader. Must be called each time a card is presented or the reader is reconnected. Reusing the connection for multiple services improves performance.

Objective-C: `-(void)connect;` / **Swift:** `func connect() -> Bool`

Throws: `ToolkitException` on error.

4.2.2. Method: Disconnect

Disconnects from the connected reader. The card can be safely removed after disconnect.

Objective-C: `-(void)disconnect;` / **Swift:** `func disconnect() -> Bool`

Throws: `ToolkitException` on error.

4.2.3. Method: GetInterfaceType

Returns the card communication interface type.

Objective-C:

```
-(int)getInterfaceType;
```

Return: `int` – 1 = Contact, 2 = NFC. **Throws:** `ToolkitException` on error.

4.2.4. Method: SetNfcAuthenticationParameters (Card Attributes)

Sets NFC authentication parameters using card attributes.

Objective-C:

```
-(void)setNfcAuthenticationParameters:(NSString *)cardNumber  
    dateOfBirth:(NSString *)dateOfBirth  
    expiryDate:(NSString *)expiryDate;
```

Parameter	Type	Description
cardNumber	NSString *	Card number of the holder.
dateOfBirth	NSString *	Date of birth in YYYYMMDD format.
expiryDate	NSString *	Card expiry date in YYYYMMDD format.

Throws: `ToolkitException` on error.

4.2.5. Method: SetNfcAuthenticationParameters (MRZ)

Sets NFC authentication parameters using the MRZ string.

Objective-C:

```
-(void)setNfcAuthenticationParameters:(NSString *)mrzData;
```

Parameter	Type	Description
mrzData	NSString *	Scanned MRZ data from the back of the ID Card.

Throws: ToolkitException on error.

4.2.6. Method: CheckCardStatus

Validates the card with VG and returns the card status.

Objective-C:

```
-(ToolkitResponse *)checkCardStatus:(NSString *)requestId;
```

Parameter	Type	Description
requestId	NSString *	Application-generated request identifier.

Return: ToolkitResponse – contains the card status. The getString method returns the XML response. **Throws:** ToolkitException on error.

4.2.7. Method: GetCSN

Returns the card serial number.

Objective-C: -(NSString *)getCSN;

Return: NSString – the card serial number. **Throws:** ToolkitException on error.

4.2.8. Method: GetCardVersion

Returns the card version number.

Objective-C: -(int)getCardVersion;

Return: int – the card version.

Value	Card Version
2	V2
3	V3
4	V4
5	V5

V5 is the latest generation card with enhanced security. Full read, PKI, and biometric operations are supported.

Throws: ToolkitException on error.

4.2.9. Method: ReadPublicData

Reads public data from the card with selectable data categories.

Objective-C:

```
-(CardPublicData *)readPublicData:(NSString *)requestId
    readNonModifiableData:(BOOL)readNonModifiableData
    readModifiableData:(BOOL)readModifiableData
    readPhotography:(BOOL)readPhotography
    readSignatureImage:(BOOL)readSignatureImage
    readAddress:(BOOL)readAddress;
```

Parameter	Type	Description
requestId	NSString *	Application-generated request identifier.
readNonModifiableData	BOOL	YES to read non-modifiable data.
readModifiableData	BOOL	YES to read modifiable data.
readPhotography	BOOL	YES to read the card holder photo.
readSignatureImage	BOOL	YES to read the handwritten signature.
readAddress	BOOL	YES to read home and work address.

Return: CardPublicData – contains requested public data. The getXmlString method returns the digitally signed XML response. **Throws:** ToolkitException on error.

4.2.10. Method: ReadPublicDataEF

Reads Elementary File data from the card.

Objective-C:

```
-(NSData *)readPublicDataEF:(int)publicDataEFTYPE
    validateSignature:(BOOL)validateSignature;
```

Parameter	Type	Description
publicDataEFTYPE	int	EF identifier (1=IDN_CN, 2=Root Cert, 3=Non-Mod, 4=Mod, 5=Photo, 6=Sig, 7=Home Addr, 8=Work Addr).
validateSignature	BOOL	YES to validate the digital signature.

Return: NSData – the EF data. **Throws:** ToolkitException on error.

4.2.11. Method: ParseEFData

Parses Elementary File data into a readable format.

Objective-C:

```
-(NSString *)parseEFData:(NSData *)efData;
```

Return: NSString – parsed EF data. **Throws:** ToolkitException on error.

4.2.12. Method: PrepareRequest (CardReader)

Establishes a secure context on the connected card.

Objective-C:

```
-(NSString *)prepareRequest:(NSString *)requestId;
```

Return: NSString – 8-byte request handle in Base64 format. **Throws:** ToolkitException on error.

4.2.13. Method: GetPkiCertificates

Reads authentication and signing certificates. PrepareRequest must be called first.

Objective-C:

```
-(CardCertificates *)getPkiCertificates:(NSString *)encodedPin;
```

Parameter	Type	Description
encodedPin	NSString *	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Return: CardCertificates – contains authentication and signing certificates. **Throws:** ToolkitException on error. getAttemptsLeft returns remaining PIN attempts.

4.2.14. Method: AuthenticatePki

Authenticates the card PIN and verifies certificate revocation. PrepareRequest must be called first.

Objective-C:

```
-(ToolkitResponse *)authenticatePki:(NSString *)encodedPin;
```

Return: ToolkitResponse – contains PKI auth status. **Throws:** ToolkitException on error.

4.2.15. Method: SignData

Signs data or hash with the signature certificate. PrepareRequest must be called first.

Objective-C:

```
-(SignatureResponse *)signData:(NSData *)input  
isInputHash:(BOOL)isInputHash  
encodedPin:(NSString *)encodedPin;
```

Parameter	Type	Description
input	NSData *	Data or hash to sign.
isInputHash	BOOL	YES if hash data; NO if plain data.
encodedPin	NSString *	ID Card PKI PIN, encrypted and Base64-encoded.

Return: SignatureResponse – contains the signature. **Throws:** ToolkitException on error.

4.2.16. Method: SignChallenge

Signs challenge data with the authentication certificate. PrepareRequest must be called first.

Objective-C:

```
-(SignatureResponse *)signChallenge:(NSData *)input
                             isInputHash:(BOOL)isInputHash
                             encodedPin:(NSString *)encodedPin;
```

Parameters are identical to SignData. **Return:** SignatureResponse. **Throws:** ToolkitException on error.

4.2.17. Method: GetFingerData

Returns finger index values and biometric reference identifiers.

Objective-C: -(NSArray *)getFingerData;

Return: NSArray Of FingerData Objects. **Throws:** ToolkitException on error.

4.2.18. Method: AuthenticateBiometricOnServer

Performs biometric authentication with VG.

Objective-C:

```
-(ToolkitResponse *)authenticateBiometricOnServer:(NSString *)requestId
                                           fingerIndex:(int)fingerIndex
                                           sensorTimeout:(int)sensorTimeout;
```

Parameter	Type	Description
requestId	NSString *	Application-generated request identifier.
fingerIndex	int	Finger index from getFingerData.
sensorTimeout	int	Timeout in seconds for the fingerprint sensor.

Return: ToolkitResponse. **Throws:** ToolkitException on error.

4.2.19. Method: AuthenticateCardAndBiometric

Validates the card and performs biometric authentication with VG.

Objective-C:

```
-(ToolkitResponse *)authenticateCardAndBiometric:(NSString *)requestId
                                           fingerIndex:(int)fingerIndex
                                           sensorTimeout:(int)sensorTimeout;
```

Parameters are identical to AuthenticateBiometricOnServer. **Return:** ToolkitResponse. **Throws:** ToolkitException on error.

4.2.20. Method: ResetPin

Resets the card PIN with biometric auth. PrepareRequest must be called first.

Objective-C:

```
-(ToolkitResponse *)resetPin:(NSString *)encodedPin  
    fingerData:(FingerData *)fingerData  
    sensorTimeout:(int)sensorTimeout;
```

Parameter	Type	Description
encodedPin	NSString *	New PIN, encrypted and Base64-encoded.
fingerData	FingerData *	Finger data from getFingerData.
sensorTimeout	int	Timeout in seconds.

Return: ToolkitResponse. **Throws:** ToolkitException on error.

4.2.21. Method: ResetPinWithoutAuthenticateBiometric

Resets the card PIN without biometric auth. PrepareRequest must be called first.

Objective-C:

```
-(ToolkitResponse *)resetPinWithoutAuthenticateBiometric:(NSString *)encodedPin;
```

Return: ToolkitResponse. **Throws:** ToolkitException on error.

4.2.22. Method: UnblockPin

Unlocks the card PIN. Involves biometric auth. PrepareRequest must be called first.

Objective-C:

```
-(ToolkitResponse *)unblockPin:(NSString *)encodedPin  
    fingerData:(FingerData *)fingerData  
    sensorTimeout:(int)sensorTimeout;
```

Parameters are identical to ResetPin. **Return:** ToolkitResponse. **Throws:** ToolkitException on error.

4.2.23. Method: ReadFamilyBookData

Reads family book data. Requires PIN auth. PrepareRequest must be called first.

Objective-C:

```
-(CardFamilyBookData *)readFamilyBookData:(NSString *)encodedPin;
```

Return: CardFamilyBookData – contains HeadOfFamily, Wives, Children. **Throws:** ToolkitException on error.

4.2.24. Method: UpdateData

Updates card data. Parameters: requestId, fileType (0-29, see FileType table in Programmer's Reference).

Objective-C:

```
-(ToolkitResponse *)updateData:(NSString *)requestId  
                           fileType:(int)fileType;
```

Return: ToolkitResponse. **Throws:** ToolkitException ON error.

4.2.25. Method: ReadData

Reads container data. Parameters: requestId, fileType.

Objective-C:

```
-(ToolkitResponse *)readData:(NSString *)requestId  
                           fileType:(int)fileType;
```

Return: ToolkitResponse. **Throws:** ToolkitException ON error.

4.2.26. Method: VerifySignature

Verifies a digital signature.

Objective-C:

```
-(void)verifySignature:(NSData *)input  
        isInputHash:(BOOL)isInputHash  
        signature:(NSData *)signature  
        certificate:(NSData *)certificate;
```

Parameter	Type	Description
input	NSData *	Data or hash corresponding to the signature.
isInputHash	BOOL	YES if hash data; NO if plain data.
signature	NSData *	The digital signature to verify.
certificate	NSData *	Certificate for signature verification.

Throws: ToolkitException if verification fails or on error.

4.3. Class: SignatureValidator

4.3.1. Constructor: SignatureValidator

Objective-C:

```
-(instancetype)initWithCert:(NSData *)cert certChain:(NSData *)certChain;
```

Parameter	Type	Description
cert	NSData *	Certificate data for response verification.
certChain	NSData *	Issuer certificate chain. Pass <code>nil</code> to skip chain validation.

4.3.2. Method: `ValidateToolkitResponse`

Validates the Toolkit XML response signature.

Objective-C:

```
-(void)validateToolkitResponse:(NSString *)toolkitResponse;
```

Throws: `ToolkitException` on error. On success, validated attributes are accessible via getter methods.

5. Advanced Digital Signature API Reference

5.1. Class: CardReader (Digital Signature Methods)

5.1.1. Method: PadesSign

Signs a PDF document (PAdES). PrepareRequest must be called first.

Objective-C:

```
-(ToolkitResponse *)padesSign:(SigningContext *)signingContext  
    pdfFilePath:(NSString *)pdfFilePath  
    signedPdfFilePath:(NSString *)signedPdfFilePath;
```

Parameter	Type	Description
signingContext	SigningContext *	Signature properties including level, mode, and PIN. Refer to the Programmer's Reference.
pdfFilePath	NSString *	Path of the input PDF.
signedPdfFilePath	NSString *	Path to store the signed PDF.

Return: ToolkitResponse. **Throws:** ToolkitException on error.

5.1.2. Method: PadesVerify

Verifies a PAdES-signed PDF.

Objective-C:

```
-(NSString *)padesVerify:(VerificationContext *)verificationContext  
    signedPdfFilePath:(NSString *)signedPdfFilePath;
```

Return: NSString – verification report in XML format. **Throws:** ToolkitException on error.

5.1.3. Method: XadesSign

Signs an XML document (XAdES). PrepareRequest must be called first.

Objective-C:

```
-(ToolkitResponse *)xadesSign:(SigningContext *)signingContext  
    xmlFilePath:(NSString *)xmlFilePath  
    signedXmlFilePath:(NSString *)signedXmlFilePath;
```

In detached mode, the detachedSignature property contains the signature. In enveloped/enveloping mode, the signature is embedded in the output file. **Return:** ToolkitResponse. **Throws:** ToolkitException on error.

5.1.4. Method: XadesVerify

Verifies an XAdES-signed XML.

Objective-C:

```
-(NSString *)xadesVerify:(VerificationContext *)verificationContext  
    signedXmlFilePath:(NSString *)signedXmlFilePath  
    signature:(NSData *)signature;
```

Pass nil for signature in enveloped/enveloping mode. **Return:** NSString – verification report. **Throws:** ToolkitException on error.

5.1.5. Method: CadesSign

Signs data (CAdES, detached only). PrepareRequest must be called first.

Objective-C:

```
-(ToolkitResponse *)cadesSign:(SigningContext *)signingContext  
    inputFilePath:(NSString *)inputFilePath;
```

Note: CAdES signing works only in detached mode.

Return: ToolkitResponse – detachedSignature contains the CAdES signature. **Throws:** ToolkitException on error.

5.1.6. Method: CadesVerify

Verifies a CAdES signature.

Objective-C:

```
-(NSString *)cadesVerify:(VerificationContext *)verificationContext  
    inputFilePath:(NSString *)inputFilePath  
    signature:(NSData *)signature;
```

Return: NSString – verification report. **Throws:** ToolkitException on error.